

1 Decision tree from Dataset to predict as a Regressor

```
[53]: import pandas as pd
      from sklearn.model_selection import train_test_split
      from sklearn.tree import DecisionTreeRegressor
      from sklearn.metrics import mean_squared_error, r2_score
```

```
[54]: # Load the dataset
      data = pd.read_csv('house_data_DT.csv')
```

```
[55]: # Ensure all column names are stripped of whitespace
      data.columns = data.columns.str.strip()
      # Check if 'Location' exists
      if 'Location' not in data.columns:
          raise KeyError("'Location' column is missing!")

      # Convert all values in the 'Location' column to strings and strip whitespace
      data['Location'] = data['Location'].astype(str).str.strip()

      # Encode the 'Location' column
      data['Location'] = data['Location'].map({'Suburb': 0, 'City': 1})

      # Check for missing or unmapped values
      if data['Location'].isnull().any():
          print("Warning: Some rows in 'Location' have unmapped or invalid values!")
          print(data[data['Location'].isnull()])
          data = data.dropna() # Drop rows with unmapped values if necessary
```

```
[56]: # Define features (X) and target (y)
      X = data[['Rooms', 'Area', 'Location']]
      y = data['Price']

      # Split the data into training and testing sets
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
      ↪random_state=42)
```

```
[57]: # Train the Decision Tree Regressor
regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X_train, y_train)
```

```
[57]: DecisionTreeRegressor(random_state=42)
```

```
[58]: # Make predictions on the test set
y_pred = regressor.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
print(f"R^2 Score: {r2}")
```

Mean Squared Error: 2500000000.0

R² Score: -10.111111111111111

```
[59]: # Function to predict price based on user input
def predict_price(rooms, area, location):
    # Encode the location input to match training data encoding
    location_encoded = 0 if location.lower() == 'suburb' else 1
    # Prepare the input data
    input_data = pd.DataFrame({
        'Rooms': [rooms],
        'Area': [area],
        'Location': [location_encoded]
    })

    # Make the prediction
    predicted_price = regressor.predict(input_data)
    print("\nPredicted Price for input (Rooms: {}, Area: {}, Location: {}): ${:
    ↵, .2f}".format(rooms, area, location, predicted_price[0]))

    # Example prediction
    predict_price(rooms=3, area=75, location='Suburb')
    predict_price(rooms=4, area=95, location='City')
```

Predicted Price for input (Rooms: 3, Area: 75, Location: Suburb): \$320,000.00

Predicted Price for input (Rooms: 4, Area: 95, Location: City): \$450,000.00